

Heart Rate & Temperature Monitoring System

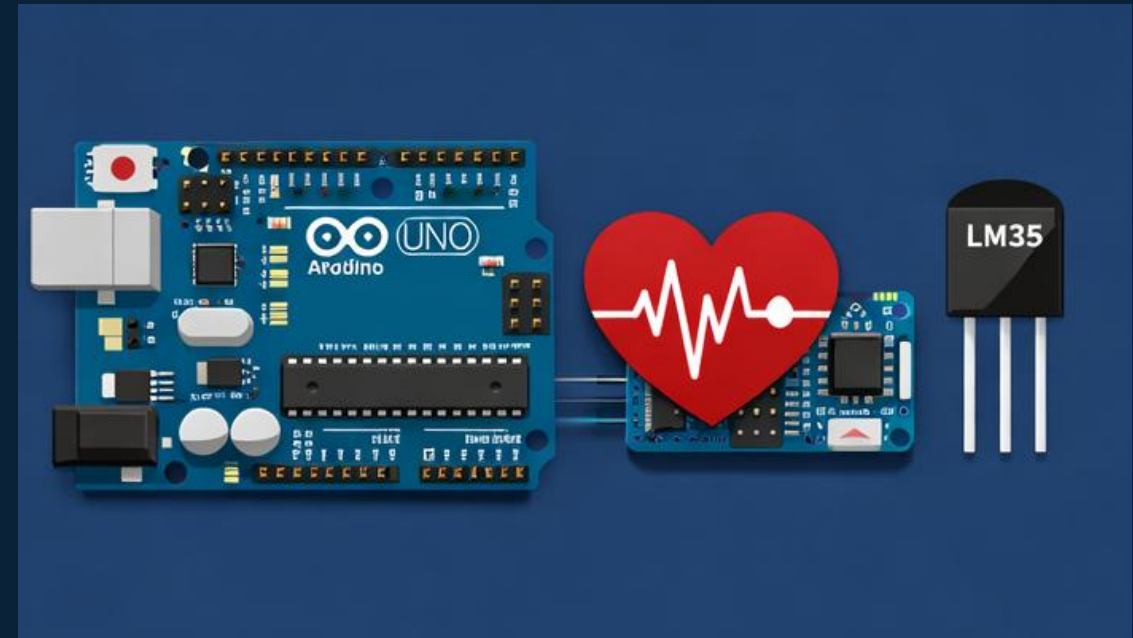
Proteus 8 Simulation · Arduino Uno · KNUST

Group 15

Department of Computer Engineering, KNUST

Members

1. Emzera Akosua Ayesu
2. Claudette Cobbah
3. Amissah-Ayiku Daniel Dromo
4. Asiseh Seginam Akosua
5. Kwabena Boateng Gyau Baffour
6. Asaah Ayebea Ayeh
7. Joshua Akwasi Donkor
8. Asare-Boateng Kwasi Boateng
9. Buafu Allan Fiifi
10. Nana Kwame Amoah Danquah



Continuous monitoring of BPM + body temperature

Table of Contents

Presentation roadmap

Speaker 2

01 Problem Statement
The health monitoring challenge

02 Methodology
Five-stage development approach

03 System Components
Hardware and instrumentation

04 Circuit & Schematic
Proteus layout and wiring

05 Code Logic
Arduino sketch and alarms

06 Demonstration
LCD, buzzer, oscilloscope

07 Results & Conclusion
Takeaways and improvements

Speaker 3

- Manual monitoring of patient vitals can be slow, error-prone, and resource-intensive.
- Patients with cardiac conditions need continuous tracking of both heart rate and body temperature.
- Critical cases include tachycardia (>100 BPM) and bradycardia (<60 BPM).
- Goal: build a low-cost embedded solution that measures, displays, and alarms in real time.

Tachycardia $\text{BPM} > 100$

Dangerously elevated heart rate requiring rapid attention.

Bradycardia $\text{BPM} < 60$

Abnormally low heart rate that may also be life-threatening.

Project answer

Proteus 8 simulation using Arduino Uno, heart sensor, LM35, 16×2 LCD, buzzer, and oscilloscope.

1

Component Selection

Choose Proteus-available parts: heartbeat sensor, LM35, LCD, buzzer, oscilloscope.

2

Schematic Design

Wire sensors to A0/A1, LCD to D2—D7, buzzer to D8, and scope to the A0 junction.

3

Code Development

Combine PulseSensorPlayground and LiquidCrystal libraries in one Arduino sketch.

4

Simulation & Testing

Compile code, load the HEX file, run Proteus, and verify LCD plus waveform output.

5

Alarm Integration

Use millis() for a non-blocking beep pattern while monitoring continues.

System Components

Six main elements of the build

Speaker 5

Heart Beat Sensor (HB1)

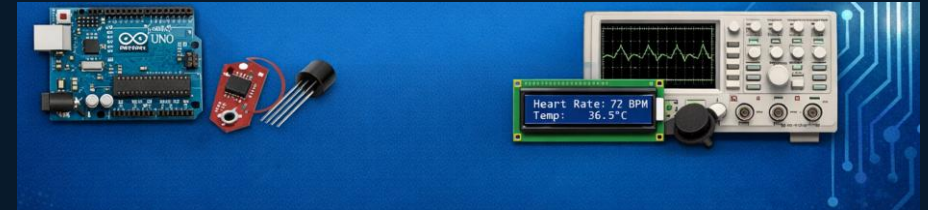
Proteus library model with internal ATmega328P + pre-compiled firmware; output to A0.

LM35 Temperature Sensor

Linear output: 10 mV/° C; ADC conversion used to compute ° C; connected to A1.

Arduino Uno

16 MHz controller executing the combined sketch and managing all I/O.



16×2 LCD (LM016L)

4-bit mode on D2—D7; VEE tied to ground for clear simulated contrast.

Buzzer

Connected to D8 and used for tachycardia/bradycardia warning.

Digital Oscilloscope

Virtual instrument; Channel A attached at heartbeat signal junction on A0.

Code & Schematic Explanation

Main loop logic and wiring summary

Speaker 6

Wiring snapshot

A0 ← Heart Beat Sensor
A1 ← LM35 via resistor
D2—D7 ← LCD in 4-bit mode
D8 ← Buzzer
Scope CH-A ← A0 junction

Temperature formula

$$\text{Temperature (}^{\circ}\text{C)} = ((\text{raw ADC} \times 5000) / 1024) / 10$$

LM35 outputs 10 mV per degree Celsius.

1

Read heart rate using `sawStartOfBeat()` and `getBeatsPerMinute()`; interrupts disabled in Proteus.

2

Read temperature from A1 and convert ADC value into degrees Celsius.

3

Set alarm state when $\text{BPM} > 100$ or $\text{BPM} < 60$.

4

Drive buzzer with `millis()`: 150 ms ON, 250 ms OFF, no blocking delay().

5

Refresh LCD every second: normal screen or HEART ATTACK warning screen.

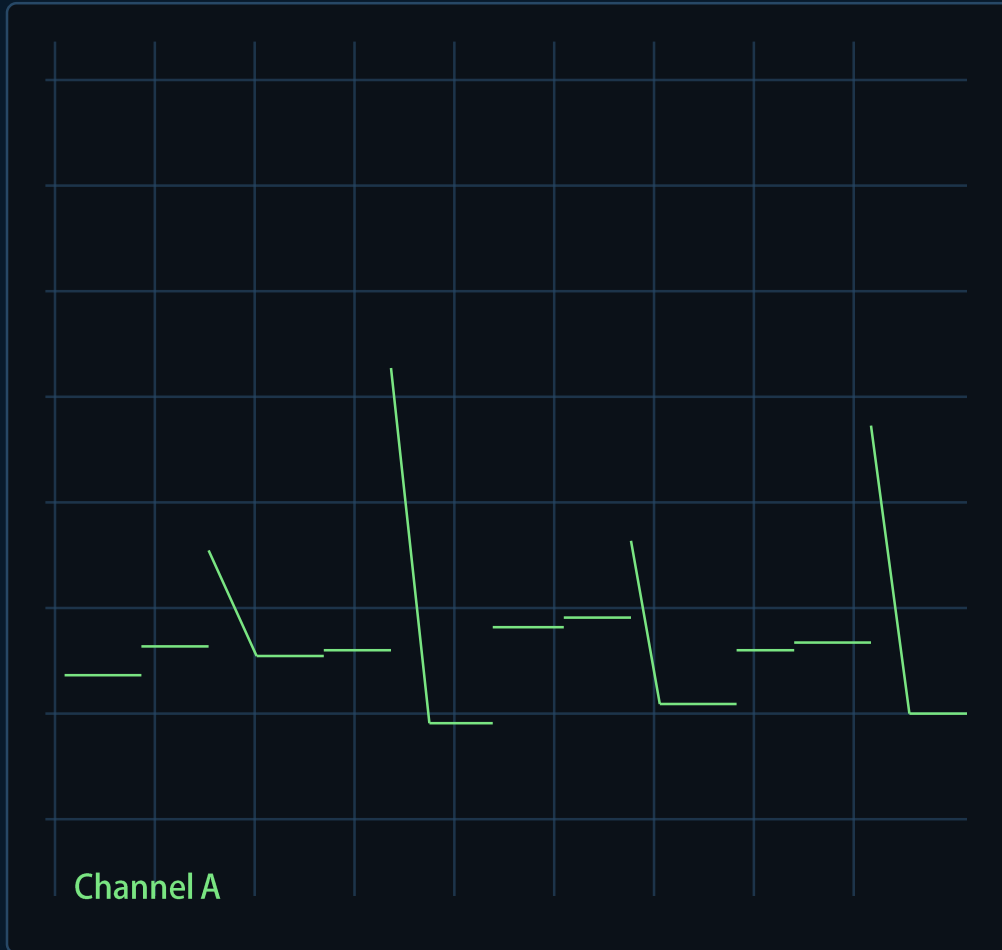
6

Keep a 20 ms stabilization delay for `PulseSensorPlayground` ADC settling.

Oscilloscope Setup

How the pulse waveform is observed

Speaker 7



- Connect Channel A at the junction between Heart Beat Sensor VccOUT and Arduino A0.
- Leave channels B, C, and D disconnected to avoid unnecessary flat baselines.
- Recommended settings: 0.2 s/div, 1 V/div, AC coupling, Trigger Source = A, Trigger Mode = Auto.
- AC coupling removes DC offset and centers the waveform for a clean display.
- Manual verification: $\text{BPM} = 60,000 / \text{period (ms)}$; e.g., 800 ms gives 75 BPM.

NORMAL MODE

BPM: 75 ♥
Temp: 36.5° C

After the splash screen, the LCD shows live BPM and temperature.

ALARM MODE

🔔 HEART ATTACK
BPM: 118 HIGH

If BPM > 100 or < 60, the buzzer starts and the LCD switches immediately.

SERIAL / SCOPE

BPM=75 | Temp=36.5 |
Alarm=OFF
BPM=118 | Temp=36.8 |
Alarm=ON

Serial Monitor logs every reading while Channel A updates in real time.

- Splash screen: "Health Monitor" and "Initialising" for the first two seconds.
- Normal display: BPM + custom heart icon on row one, temperature on row two.
- Alarm display: bell icon, HEART ATTACK message, and HIGH/LOW BPM status.
- Buzzer uses a repeating beep pattern while the scope waveform continues updating.

Results

- The system simultaneously reads and displays heart rate and body temperature in real time.
- The alarm correctly identifies tachycardia and bradycardia and triggers both LCD warning and buzzer.
- Oscilloscope waveform repeats clearly and BPM from the waveform matches LCD output.

Key Takeaways

- Real-time dual-sensor monitoring is feasible on a low-cost Arduino platform.
- Non-blocking `millis()` logic is essential when monitoring and alarming must happen together.
- Simulation library internals matter: the heartbeat sensor behaves like a microcontroller-driven source.
- Readable scope output depends heavily on AC coupling, volts/div, and time/div tuning.

Conclusion, Challenges & Improvements

Final assessment and future work

Speaker 10

Conclusion

The project successfully simulates a patient-vitals monitor that measures BPM and temperature, displays readings on an LCD, warns on dangerous BPM, and visualises the pulse waveform in Proteus 8.

Challenge

S

- Heartbeat sensor uses internal firmware, so BPM is not easily adjustable in simulation.
- PulseSensor interrupts fail in Proteus; polling mode is required.
- Scope time base expects seconds, so 200 ms must be entered as 0.2.
- Extra filter/amplifier stages caused phase shift and false double-beat detection.

Future Improvements

- Replace the sensor model with a 1.25 Hz signal generator for more realistic PPG testing.
- Add Wi-Fi or Bluetooth for remote patient monitoring.
- Integrate an SpO₂ sensor and timestamp readings with an RTC module.
- Use distinct buzzer tones for tachycardia vs bradycardia.

Thank you for your attention.